

PORTFOLIO

# Edward E. Gemadzi

Full-stack developer building secure web platforms

ACCRA, GHANA

SELECTED WORK

|    |                            |                           |
|----|----------------------------|---------------------------|
| 01 | GroupFund                  | Alumni association, Ghana |
| 02 | Pretty in Pink Brunch 2026 | Pretty in Pink Brunch     |
| 03 | Trading Desk               | Personal project          |
| 04 | APSU '16                   | APSU '16                  |

## — About

I design and build production web applications end to end: interface, backend, database, payments and deployment. I put security first, with encrypted personal data, role-based access and audited admin tools, and I ship with automated tests and real monitoring.

## — Skills

Next.js

TypeScript

React

Supabase / Postgres

Payments (Paystack)

Security and data protection

UI/UX design

Vercel and Cloudflare

tRPC

Prisma

Double-entry accounting

Neon Postgres

## — Contact

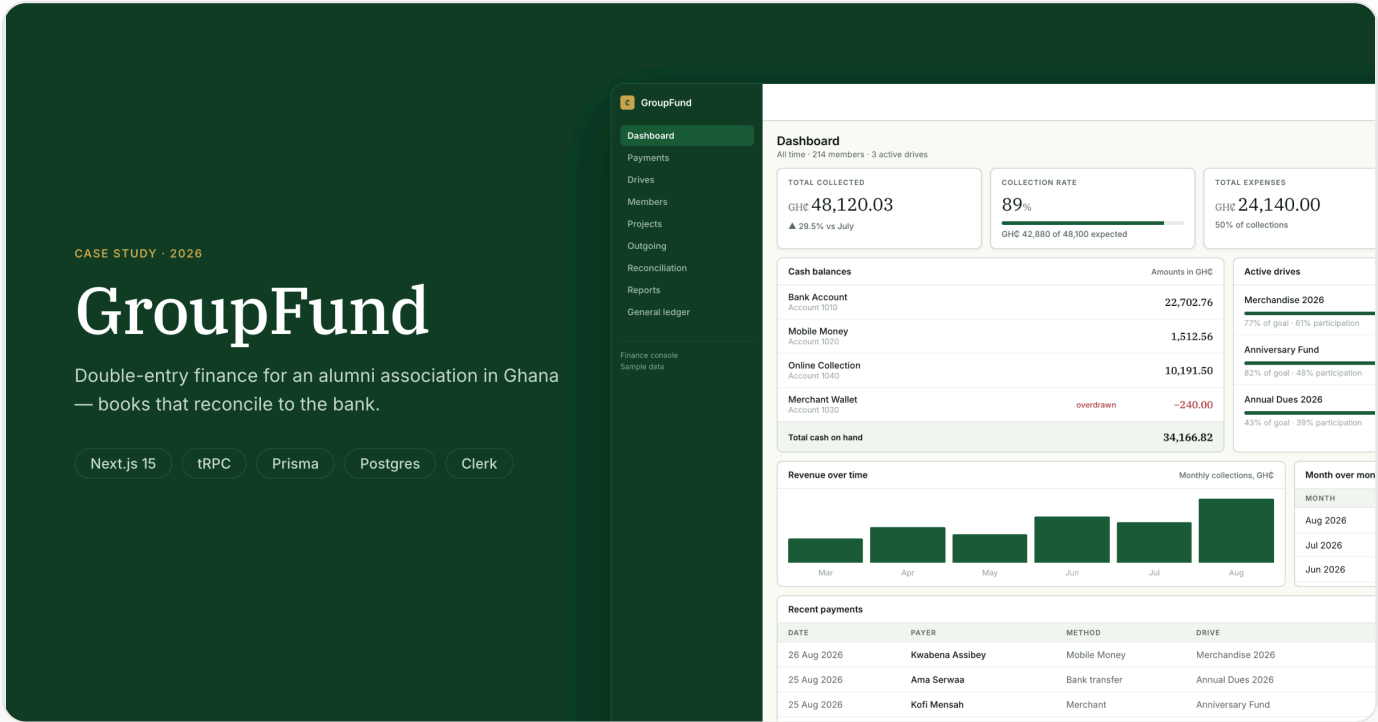
BASED IN

Accra, Ghana

# GroupFund

Double-entry finance platform for an alumni association

| CLIENT                    | ROLE                                                             | TIMELINE       | YEAR |
|---------------------------|------------------------------------------------------------------|----------------|------|
| Alumni association, Ghana | Design, full-stack build, accounting model, security, deployment | Apr – Sep 2026 | 2026 |



A finance platform for an alumni association that collects dues, runs merchandise and event ventures, and answers to its members for every cedi. Payments arrive by bank transfer, mobile money and online checkout; the committee records and verifies them, and the system posts each one into a real double-entry general ledger. Treasurers get a trial balance, balance sheet and income statement that agree, instead of a spreadsheet nobody can reconcile.

|                                                         |                                         |                                             |                                                      |
|---------------------------------------------------------|-----------------------------------------|---------------------------------------------|------------------------------------------------------|
| 21                                                      | 73                                      | 9                                           | 144                                                  |
| double-entry ledger accounts; reports agree to the bank | automated tests on the accounting rules | committee roles with least-privilege access | server procedures, each checking its own permissions |

## The challenge

- Money arrived through four separate channels and was tracked in spreadsheets that never agreed with the bank statement.
- The group collects money it does not own — merchandise paid for in advance, funds raised to pass on to another year group — and counting that as income overstated what the association actually had.
- Committee roles change every year, so access had to be narrow and auditable rather than one shared login.
- Members wanted to see where their contributions went without the committee hand-building reports.

## What I built

- Every payment posts a matching journal entry, so the cash view and the accounting view can never drift apart.
- Money held on behalf of someone else posts to a liability account instead of income, and only becomes revenue when the obligation is discharged.
- Nine committee roles with least-privilege access, segregation of duties on every payment, and an audit log on sensitive actions.
- Reports generated from the ledger itself: trial balance, balance sheet, income statement, debtors and creditors ageing, budget versus actual.

## Key features

### 01

#### Double-entry general ledger

21 accounts across assets, liabilities, equity, income and expenses. Every payment, charge and payout posts a balanced journal entry.

### 02

#### Payment drives

Seven kinds of collection — dues, pledges, merchandise, donations, event fees — each with its own target, deadline and posting rules.

### 03

#### Three-step payment approval

Recorded, verified, then acknowledged by three different people. The same person cannot do two of the three.

### 04

#### Bank reconciliation

Payment alerts are parsed from bank emails into a queue, matched to a member and drive, or dismissed with a reason.

### 05

#### Funds held in trust

Money collected to pass on is held as a liability per venture, so the dashboard separates what the group owns from what it holds.

### 06

#### Member ledger and standings

What each member has given and still owes, plus house and class leaderboards generated from the same figures as the reports.

### 07

#### Investor ventures

Members can back a venture; capital and profit share are tracked separately and settled in cash or offset against what they owe.

### 08

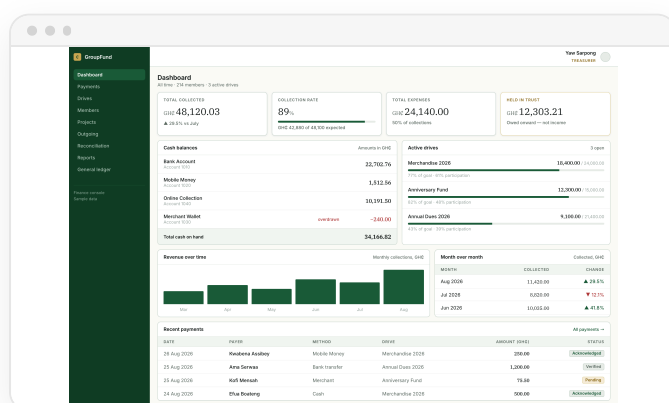
#### Reports and exports

Trial balance, balance sheet, income statement, ageing and cash-flow forecast, exportable to CSV, Excel and PDF.

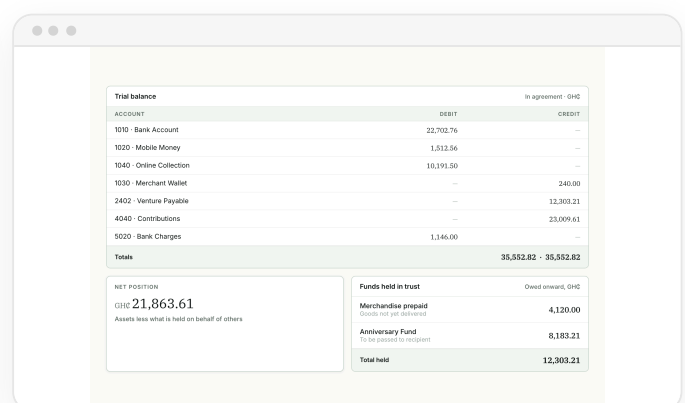
### 09

#### Announcements

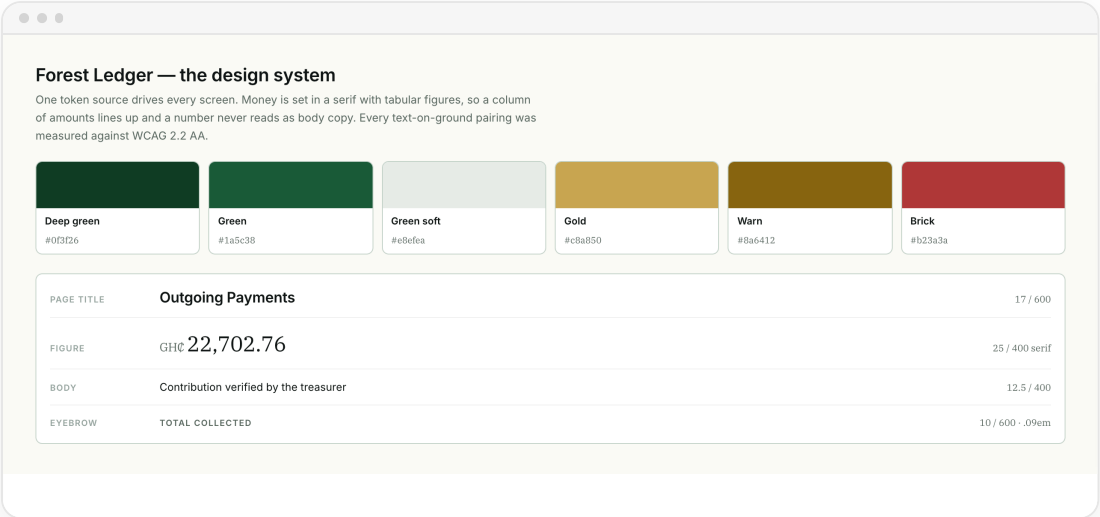
Email and SMS to filtered member groups, with personal payment links generated per recipient.



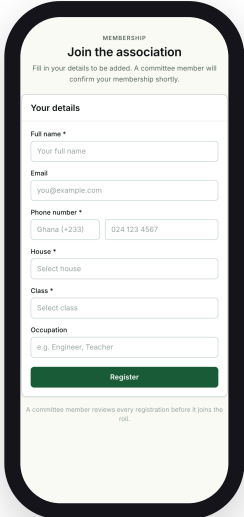
Treasurer dashboard: cash position, drives and the payments register (sample figures)



Trial balance in agreement, and funds held for others kept off income (sample figures)



Design system: one token source drives every screen



Registration on a phone

## Security & quality

- Nine committee roles enforced in every server procedure, not just the interface.
- Recording, verifying and acknowledging a payment must be done by three different people.
- Corrections are reversals, never edits: both sides stay on the record for audit.
- Signed payment webhooks, verified inbound email, and Zod validation on every input.
- 73 automated tests on the accounting rules; type checks and a full build gate every release.

## Stack



# Pretty in Pink Brunch 2026

Event registration, payments and QR check-in platform

**CLIENT**

Pretty in Pink Brunch

**ROLE**

Design, full-stack build, security, deployment

**TIMELINE**

Sept – Oct 2026

**LIVE**

[prettyinpink.org.uk](https://prettyinpink.org.uk)

SATURDAY 17 OCTOBER 2026 · 12PM – 5PM

## PRETTY IN PINK BRUNCH

*A luxury brunch experience for breast cancer awareness*

Country Club, Trasacco Valley · Accra



A complete event platform for a charity brunch in Accra: guests register, pay a minimum contribution through Paystack, receive a personal QR pass by email, and are checked in at the door from a phone. Organisers run everything from a secure dashboard: guests, seating, VIP places, emails, donations and the site's own content.

**Live**

at [prettyinpink.org.uk](https://prettyinpink.org.uk)

**291**

automated tests

**5**

staff roles with least-privilege access

## The challenge

- Every guest had to register and contribute at least GHS 200 before receiving an entry pass.
- The organisers needed a paid VIP tier with a hard limit of 50 places and upgrades for existing guests.
- Guests' personal data had to be protected properly, not just stored.
- The organising team needed to change website text, seat guests and email guests without a developer.

## Key features

**01**

### Registration and contribution gate

4 steps with a GHS 200 minimum. Rejects disposable or mistyped emails and blocks duplicate sign-ups.

**02**

### Paystack payments

Amounts set on the server, every payment checked with Paystack, signed webhooks, and automatic receipts.

**03**

### Paid VIP tier

GHS 1,000+, capped at 50 places, with 30-minute checkout holds and upgrades for Standard guests.

**04**

### Digital QR passes

A personal pass with table, seat and VIP perks, issued only once payment is confirmed.

**05**

### Door check-in

Scan or search from a phone, take payment at the door, and catch duplicates. Attendance updates live.

**06**

### Seating and tracking links

Tables with auto-fill seating, plus tracking links and QR codes for each campaign.

**07**

### Guest emails

Email groups or hand-picked guests from templates, with consent checked at send time.

**08**

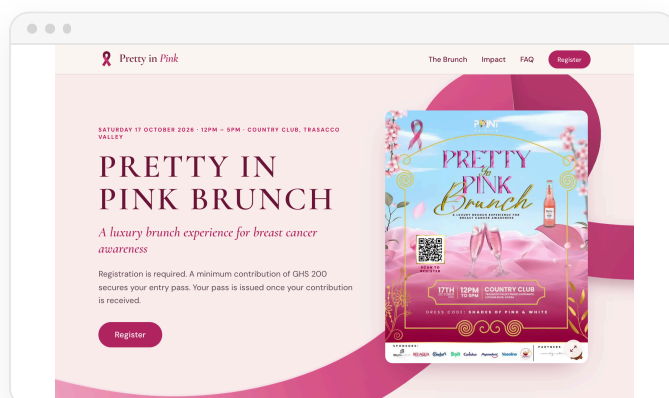
### Content editor

Organisers edit website text, sponsors and VIP perks themselves, with history and undo.

**09**

### Live donations

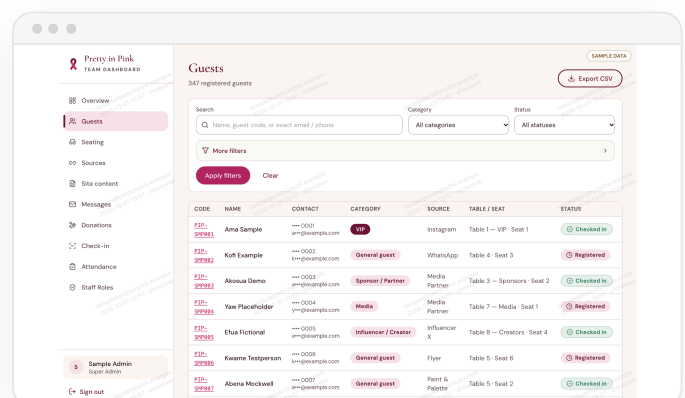
New Paystack payments appear in the dashboard automatically, with no refresh.



Home page: hero with the event flyer

## What I built

- A 4-step registration flow with Standard and VIP options, email checks and duplicate blocking.
- Paystack payments checked on the server, with signed webhooks, so a pass is issued only after payment is confirmed.
- Personal QR passes and a phone-friendly check-in screen that can also take payment at the door.
- A role-based admin dashboard with two-step sign-in, live payment updates and a built-in content editor.



Admin: guest list with masked contact details (sample data)

Your experience: Standard · [Change](#)

Your contribution will help support initiatives, including free breast-health screening, breast-health education, school supplies, and community support.

☒ **Contribute now**  
Pay securely online with card or mobile money. Your entry pass is issued as soon as your payment is confirmed.

☐ **Contribute later**  
Your entry pass is issued once your contribution is received — online before the event or at the registration point on the day.

Amount (GHS) \*

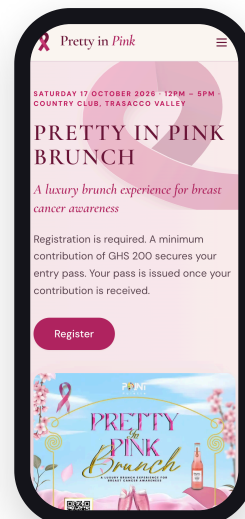
GHS 1,000 GHS 700 **GHS 500** GHS 200

[Other](#)

🔒 You will pay on Paystack's secure page. Your email is shared with Paystack to process the payment and send your receipt.

[Back](#) [Continue to secure payment](#)

Registration: contribution step



Mobile home page

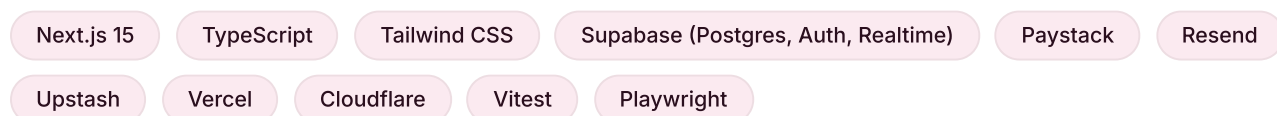


Digital QR pass (sample data)

## Security & quality

- Personal data encrypted with AES-256-GCM, with keyed hashes for searching without decrypting.
- Staff roles enforced in every API route and by database access rules; admins must use an authenticator app to sign in.
- Contact details masked by default, every reveal recorded in an audit log, staff screens watermarked, and idle sessions signed out automatically.
- Strict security headers (CSP and HSTS) and rate limiting on public forms.
- 291 automated tests; type checks and a full build must pass before every release.

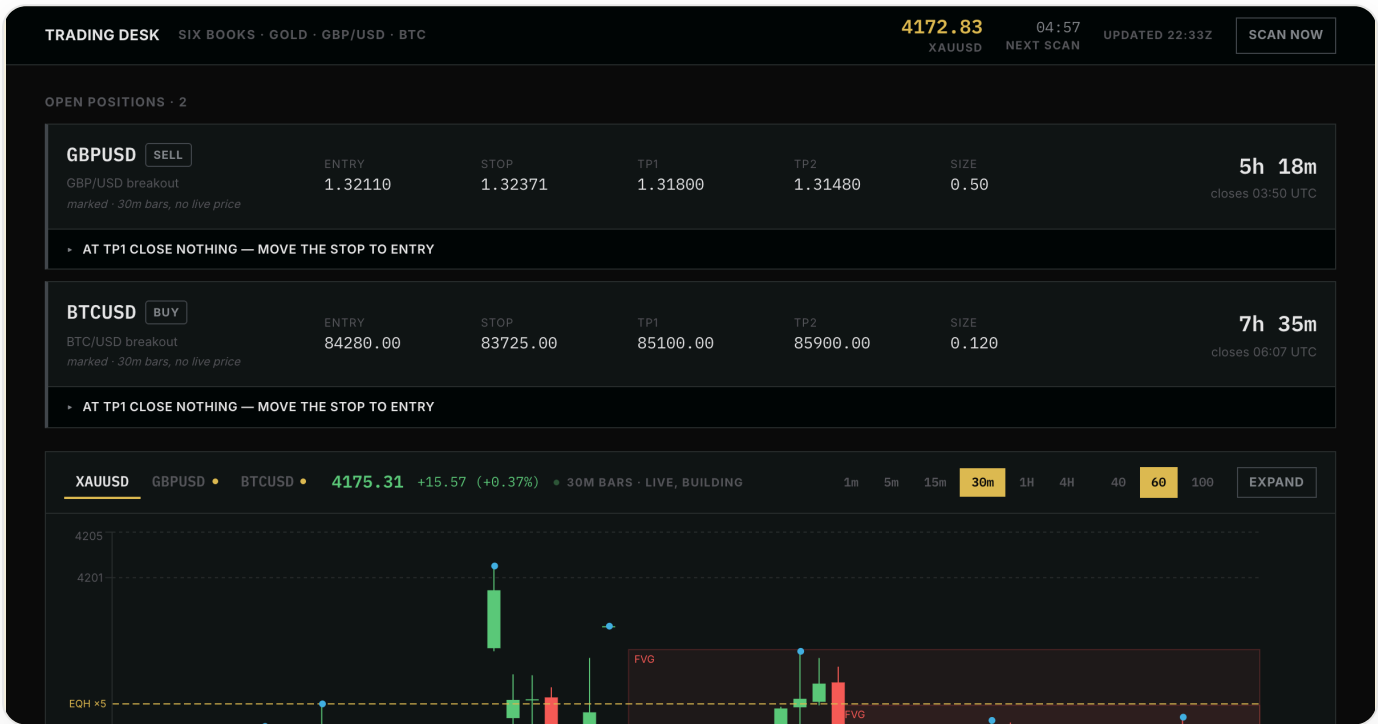
## Stack



# Trading Desk

Rule-based trading signals, live position tracking and AI review for gold, GBP/USD and Bitcoin

| CLIENT           | ROLE                                                     | TIMELINE       | YEAR |
|------------------|----------------------------------------------------------|----------------|------|
| Personal project | Product, strategy research, full-stack build, operations | Jun – Oct 2026 | 2026 |



A private trading desk for gold, GBP/USD and Bitcoin that runs six rule-based strategies around the clock on a dedicated Mac. It sends the trader Telegram alerts with exact levels and the next action to take, and tracks every open position from a live price stream. Every strategy, and every change to one, is tested on three years of history against fixed pass-or-fail checks before it goes live. AI models give an advisory second opinion that the code checks against the facts before anyone sees it.

|                                                                            |                                                        |                                                                                   |                                                                                  |
|----------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| 6                                                                          | 2.5s                                                   | 6                                                                                 | 8                                                                                |
| strategy books across three markets, each tested on three years of history | between position checks, driven by a live price stream | daily automated checks that the live strategy code still matches the tested rules | background services, running 24 hours a day, five days a week on a dedicated Mac |

## The challenge

- Trades are placed by hand, so every alert has to say exactly what to do and when, or the trade is missed.
- Signals arrived late: a scan every five minutes and a data cache out of step with the market meant entries landed 7.1 minutes after the price bar that triggered them, on average.
- A free market-data plan of 800 requests a day had to serve the charts, the strategy scans and the research.
- Most trading ideas look convincing on a chart and fall apart when tested on data that wasn't used to choose them.

## Key features

**01**

### Six strategy books, three markets

Range breakouts, a Bollinger band ride, a Donchian channel and a swing system on gold, GBP/USD and Bitcoin.

**02**

### Live price stream and shared bar store

WebSocket ticks build 1-minute to 4-hour bars once. Bars the feed didn't fully watch are never used.

**03**

### Two-stage Telegram alerts

A warning while a setup forms, then the entry with its levels and next action, then every exit.

**04**

### AI second opinion, checked in code

Gemini reviews each signal; code checks its verdict against computed facts. Ollama and FinBERT back it up.

**05**

### Position tracking at stream speed

Stops, targets and trailing stops checked every 2.5 seconds from an in-memory position cache.

**06**

### Trading console

Built from a Claude Design spec: next action per position, live multi-timeframe chart, phone layout.

**07**

### Official economic calendar

Release times from the Fed, BLS, BEA and Bank of England. Fed dates parsed by code, changes flagged.

**08**

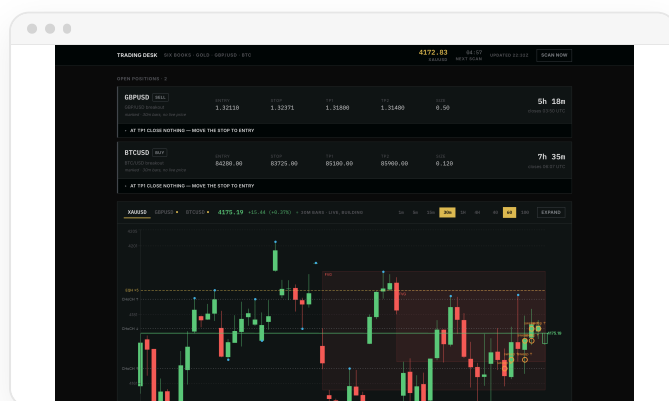
### Research pipeline with fixed pass marks

Held-out data, resampling, cost sweeps and best-trade checks. A weekly job proposes, never applies.

**09**

### Built to keep running through outages

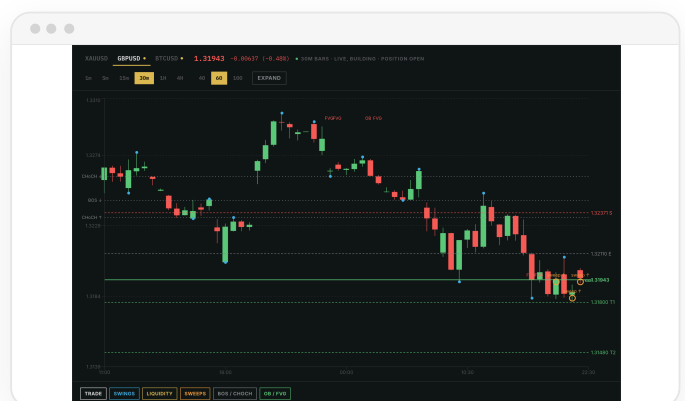
Falls back to live-feed bars when the provider is down; a daily budget protects the data allowance.



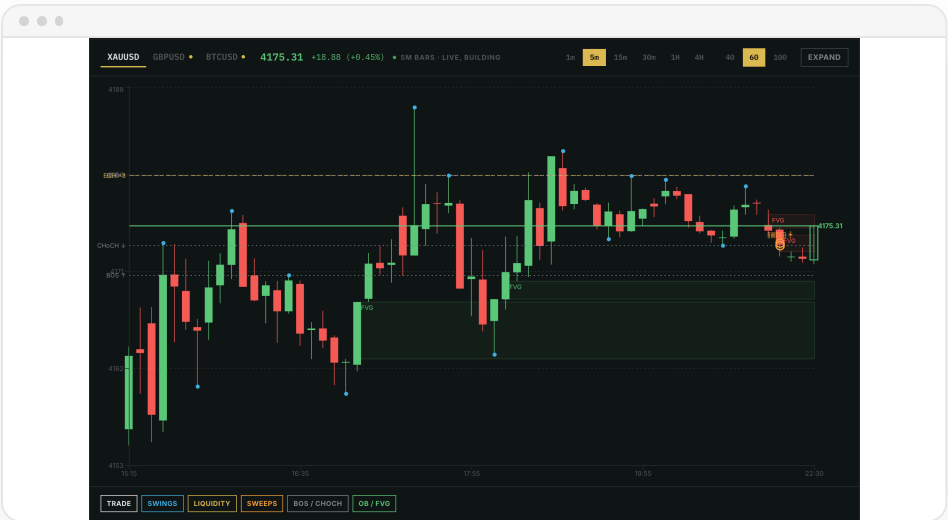
Console: open positions with their next action, above the live chart (sample data)

## What I built

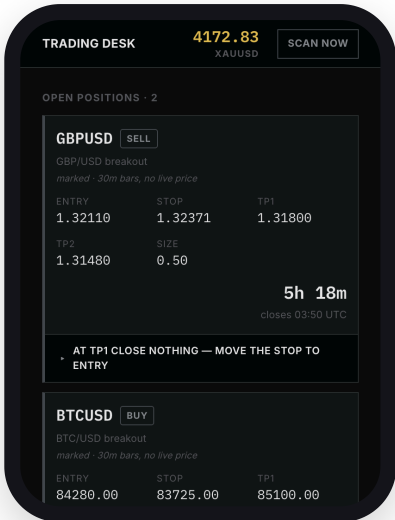
- Alerts in two stages: a warning while a setup is forming, then the entry with its levels and a one-line instruction.
- Scans run every minute and fire within seconds of a price bar closing, and open positions are checked against a live price stream every 2.5 seconds.
- One live price feed builds every chart timeframe once. The charts and strategies read the same data, and the paid provider only fills in history and repairs gaps.
- A fixed research method for every change: fit the rule on the first half of three years, confirm it on the second half, and require statistical evidence before it ships.



A position's entry, stop and targets drawn on the live GBP/USD chart (sample data)



Gold on 5-minute bars with swing, liquidity and structure overlays (sample data)



The same console on a phone (sample data)

## Security & quality

- API keys stay on the server and are never logged.
- Endpoints accept only the traded markets and validate every price and timestamp.
- AI never places or sizes a trade: its verdicts are advisory and checked in code.
- Calendar pages come from a fixed list of official sites and are treated as data, never instructions.
- Six checks run every morning to prove the live code still matches the tested rules.

## Stack

Next.js 16

React 19

TypeScript

Prisma 7

Neon Postgres

Zod

Recharts

Tailwind CSS 4

Gemini API

Ollama

FinBERT

Finnhub WebSocket

Telegram Bot API

macOS launched

# APSU '16

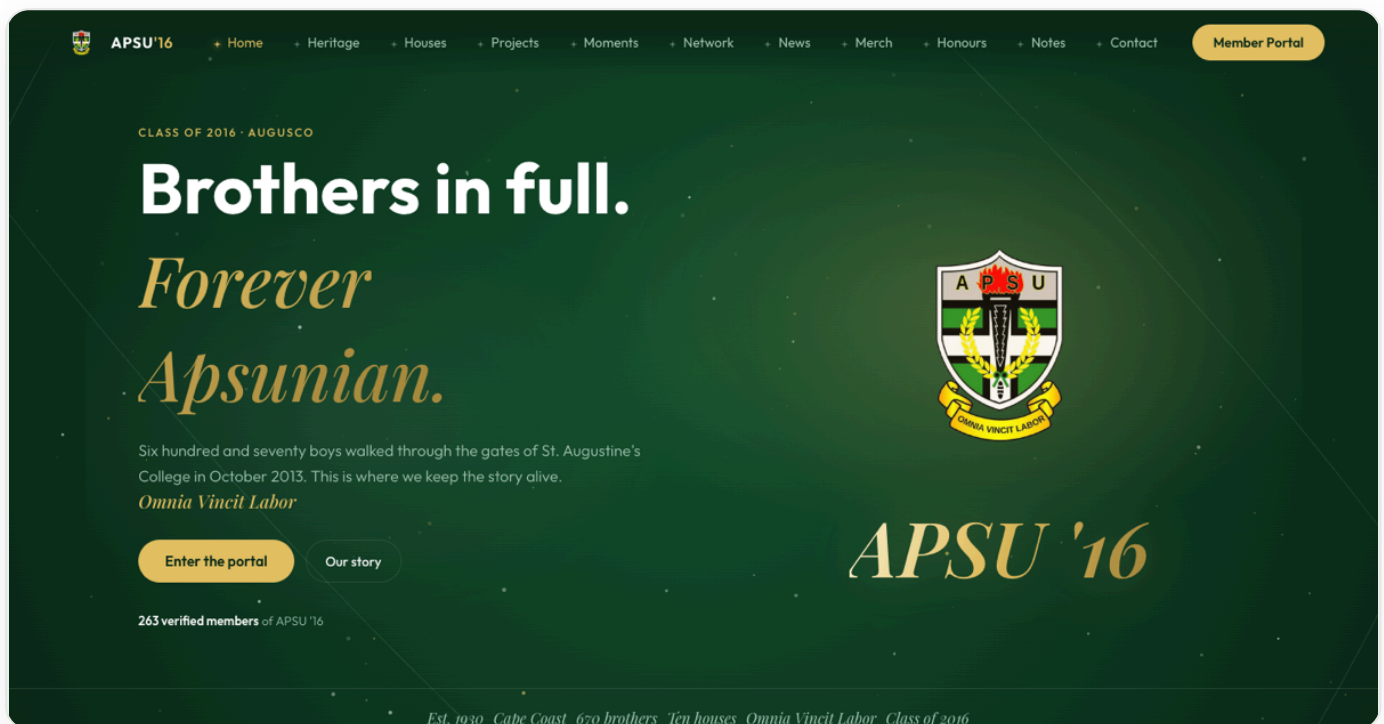
Alumni platform with a private member portal, event payments and moderated community content

CLIENT  
APSU '16

ROLE  
Design, full-stack build, security, deployment

TIMELINE  
2026 – ongoing

LIVE  
apsu16.com



A home for the Class of 2016 of St. Augustine's College, Cape Coast. The public site tells the year group's story and gives every member a house to belong to. Behind it, a private portal lets verified members find each other, RSVP to events, pay dues and post updates. The association's executives run the site themselves: they edit content, review submissions and track who is attending.

**Live**

at apsu16.com

**263**

verified members

**22%**

less JavaScript per public page (352 kB down to at most 275 kB)

**10**

house pages, one for each school house

## The challenge

- More than 250 members needed a private place to find each other, but the member list had to stay hidden from the public.
- Events needed RSVPs where members choose options and pay straight away through the association's existing payment drive.
- Executives needed to review honours nominations and member updates without asking a developer.
- Most members use phones on slow mobile connections, so every page had to be light and fast.

## Key features

01

### Private member portal

A member directory, events, dues, profile and messages in one place, designed for phones first.

02

### Event RSVPs with options

Members tick the items they want and see the total. The server sets the price, then hands off to GroupFund to pay.

03

### Honours and class notes

Members nominate brothers for awards and post updates. Executives review both in the app and members are notified.

04

### Self-service content

Admins change the site's text and images themselves. Every admin action goes into an audit log.

05

### Opening animation

Particles burst like fireworks and form the wordmark. Plays once per visitor, with a reduced-motion fallback.

06

### Ten houses

Each school house has its own page, so every member belongs to a smaller group within the year.

07

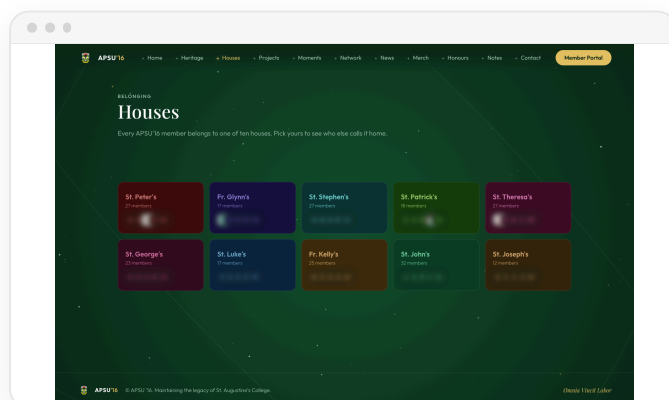
### Passwordless sign-in

Members sign in with a one-time code sent by email. The membership check runs on the server, with rate limiting.

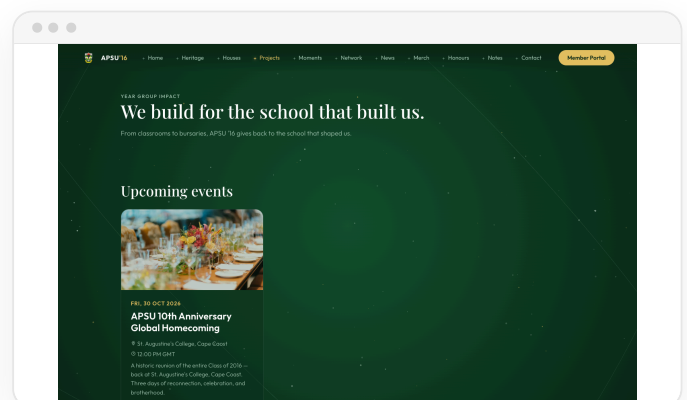
08

### Attendee lists

Admins see who confirmed and what each person chose, then export the list to a spreadsheet.



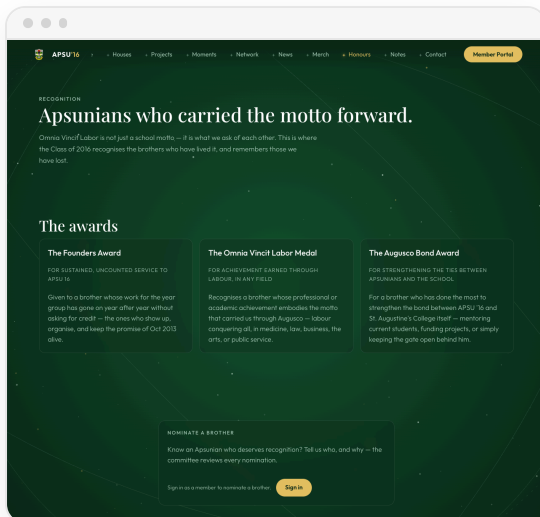
Houses: every member belongs to one of ten (member photos blurred)



Projects and upcoming events

## What I built

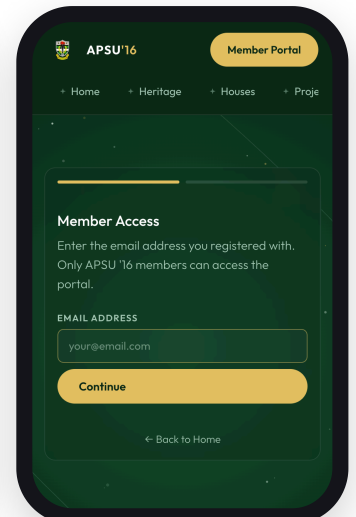
- A public site and a separate member portal. Members sign in with a one-time code, and access rules in the database protect member data.
- Event packages with selectable options. The server calculates the total and sends the member to the existing GroupFund payment drive.
- Admin panels for nominations, class notes, events and site content, with in-app notifications and an audit log.
- A redesign that cut the JavaScript on every public page, plus a full security hardening pass.



Honours: the year group's awards and nominations



Opening animation: particles form the wordmark



Member sign-in with a one-time code

## — Security & quality

- Row-level security on member data. Sensitive database functions run on the server only.
- Payment amounts calculated on the server. The browser only sends which package was chosen.
- Membership lookups moved into a rate-limited server route, to stop scripted email checks.
- Uploads checked by file contents, not file name. Spreadsheet exports guarded against formula injection.
- Security headers (CSP, HSTS, no framing) and error monitoring with Sentry.

## — Stack

Next.js 15

React 19

TypeScript

Supabase (Postgres, Auth, Storage, Realtime)

GSAP

HTML Canvas

GroupFund (mobile money)

Sentry

Vercel

CONTACT

**Let's build something  
together.**

BASED IN    Accra, Ghana